

8.4

Преобразование (приведение) чисел

00:00–00:16

Введение

Привет!

В этой теме поговорим о преобразованиях или приведении чисел разных типов к числам других типов и познакомимся с такими понятиями, как явное и неявное преобразование.

00:16–03:07

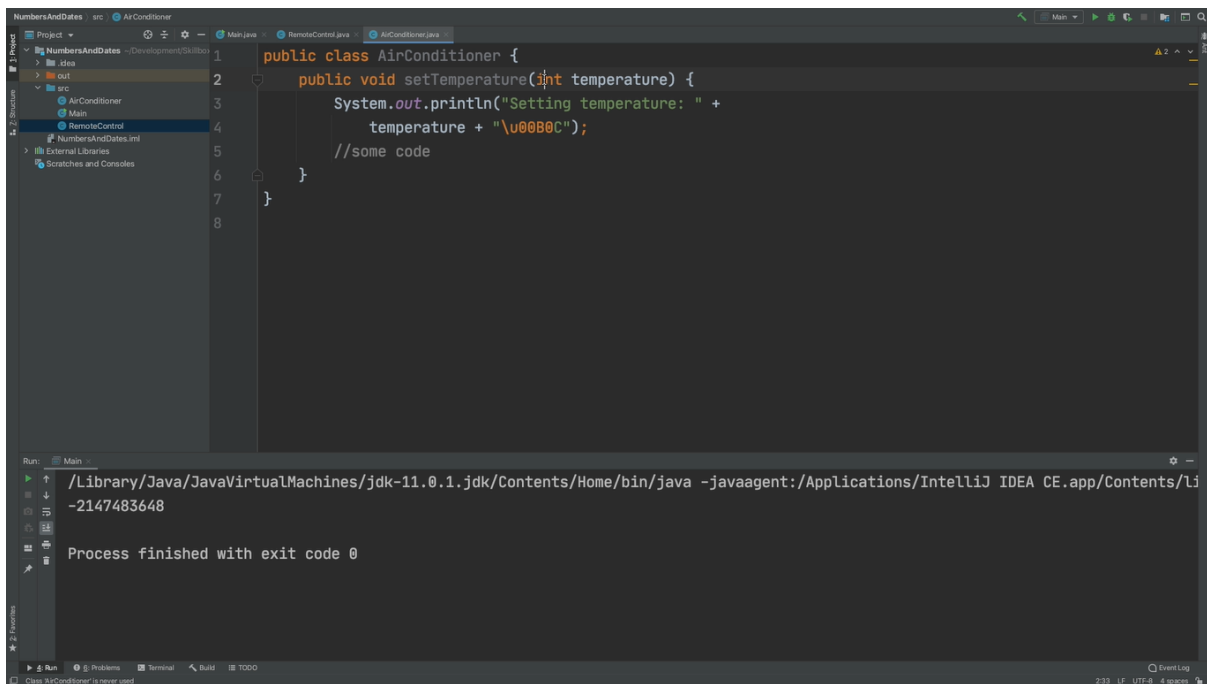
Преобразование чисел разных типов. Явное преобразование

Java — строго типизированный язык, то есть типы всех переменных всегда задаются в явном виде и не меняются. Это очень удобно, поскольку вы всегда знаете, какие типы данных где используются.

Типы используемых в коде переменных контролируются автоматически — такая строгая типизация приводит к высокой предсказуемости в работе, и мы всегда точно знаем, как будет работать тот или иной код.

Посмотрим на примерах.

Например, вы разрабатываете программу, которая управляет кондиционером. У вас есть класс `AirConditioner`, у которого есть метод `SetTemperature`, на вход которого можно передать определённую температуру, и кондиционер начинает выдувать воздух именно с этой температурой. Температуру на устройство нагрева воздуха можно передавать только в виде целого числа, поэтому параметр метода `SetTemperature` имеет тип `int`, то есть целого числа.



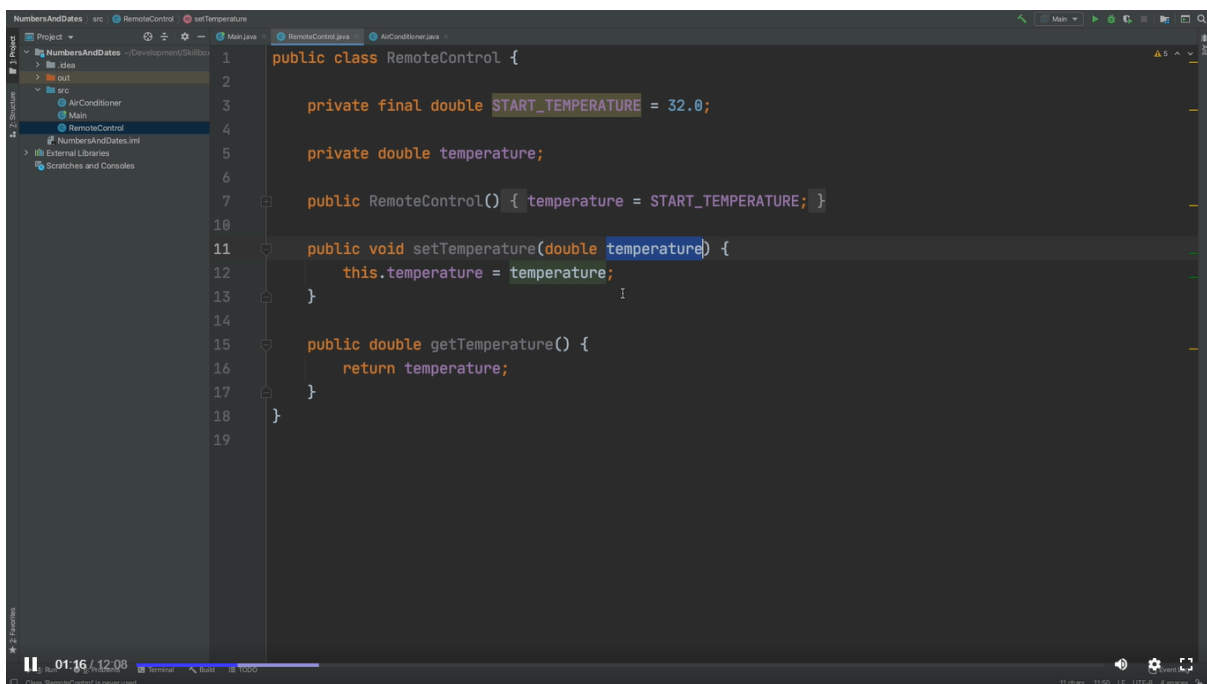
The screenshot shows an IDE window with a project named "NumbersAndDates". The "src" folder contains two files: "AirConditioner.java" and "RemoteControl.java". The "AirConditioner.java" file is open, showing the following code:

```
1 public class AirConditioner {
2     public void setTemperature(int temperature) {
3         System.out.println("Setting temperature: " +
4             temperature + "\u00B0C");
5         //some code
6     }
7 }
8
```

The "Run" tab at the bottom shows the execution output:

```
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
-2147483648
Process finished with exit code 0
```

Также у вас есть пульт управления кондиционером, представленный классом RemoteControl, в который вы можете передавать температуру в виде числа с плавающей точкой. К примеру, пульт и кондиционер были разработаны различными производителями, и они не согласовали друг с другом формат передачи данных.

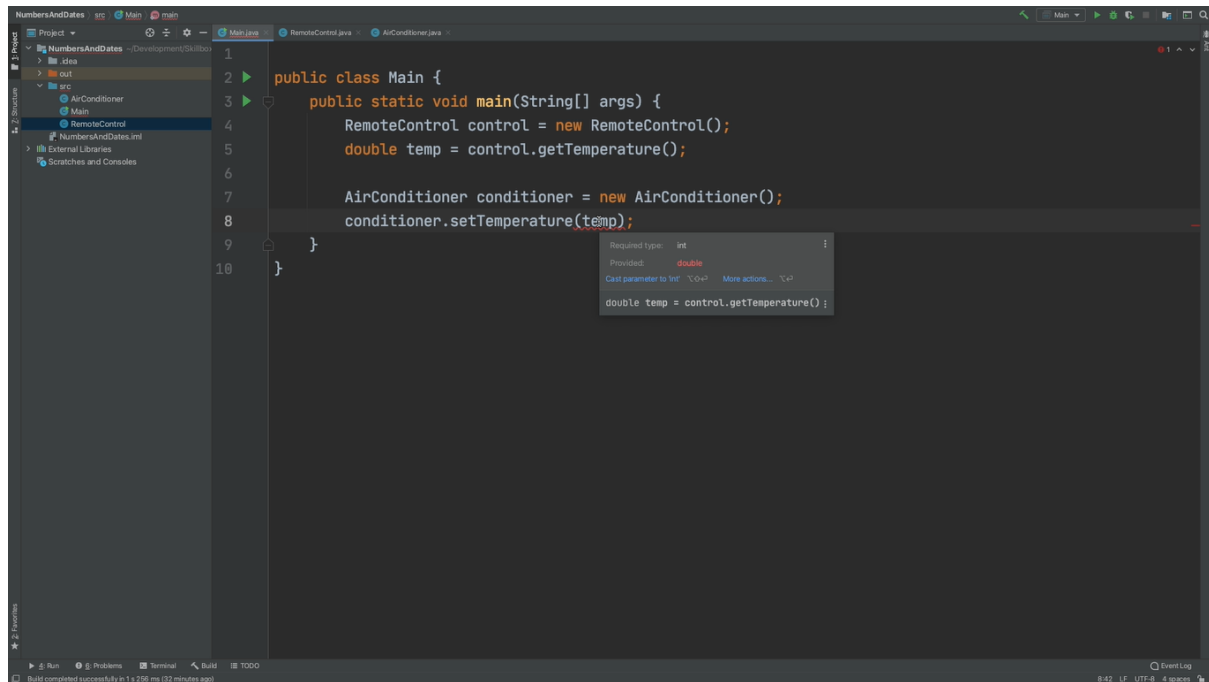


The screenshot shows the same IDE window with the "RemoteControl.java" file open. The code is as follows:

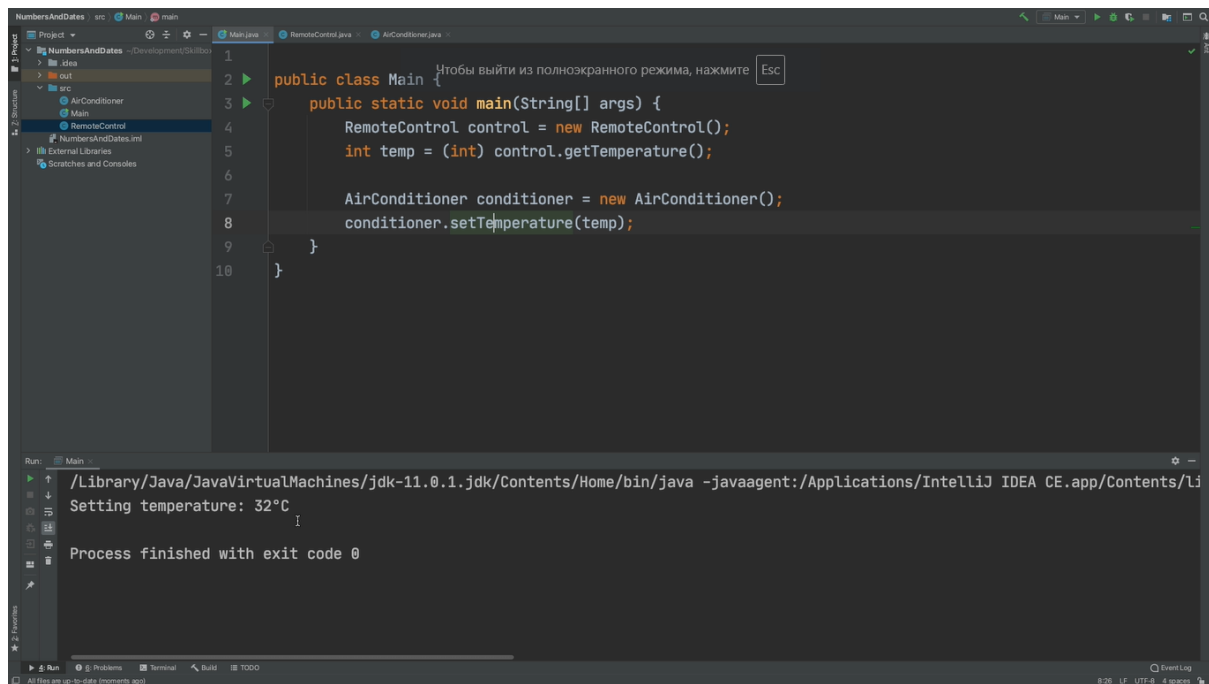
```
1 public class RemoteControl {
2
3     private final double START_TEMPERATURE = 32.0;
4
5     private double temperature;
6
7     public RemoteControl() { temperature = START_TEMPERATURE; }
8
9
10
11     public void setTemperature(double temperature) {
12         this.temperature = temperature;
13     }
14
15     public double getTemperature() {
16         return temperature;
17     }
18 }
19
```

Вы пишете код: создаёте сначала объект класса RemoteControl и получаете из него температуру. Затем создаёте объект класса AirConditioner и передаёте ему эту температуру.

У вас возникает ошибка. Если навести курсор мыши на подчёркнутое слово temp, здесь будет написано "required type: int, provided type: double", то есть "требуемый тип данных: int, предоставленный тип: double".



В этом случае можно преобразовать число из типа double в int. Для этого нужно поменять тип этой переменной на int и в скобках указать тот тип, к которому мы хотим преобразовать это число:



Это первый вид преобразования типов, и он называется **явным**, поскольку вы в явном виде указываете тот тип, к которому нужно преобразовать число.

Преобразование чисел разных типов

Явное преобразование:

```
int value = (int) doubleValue;
```

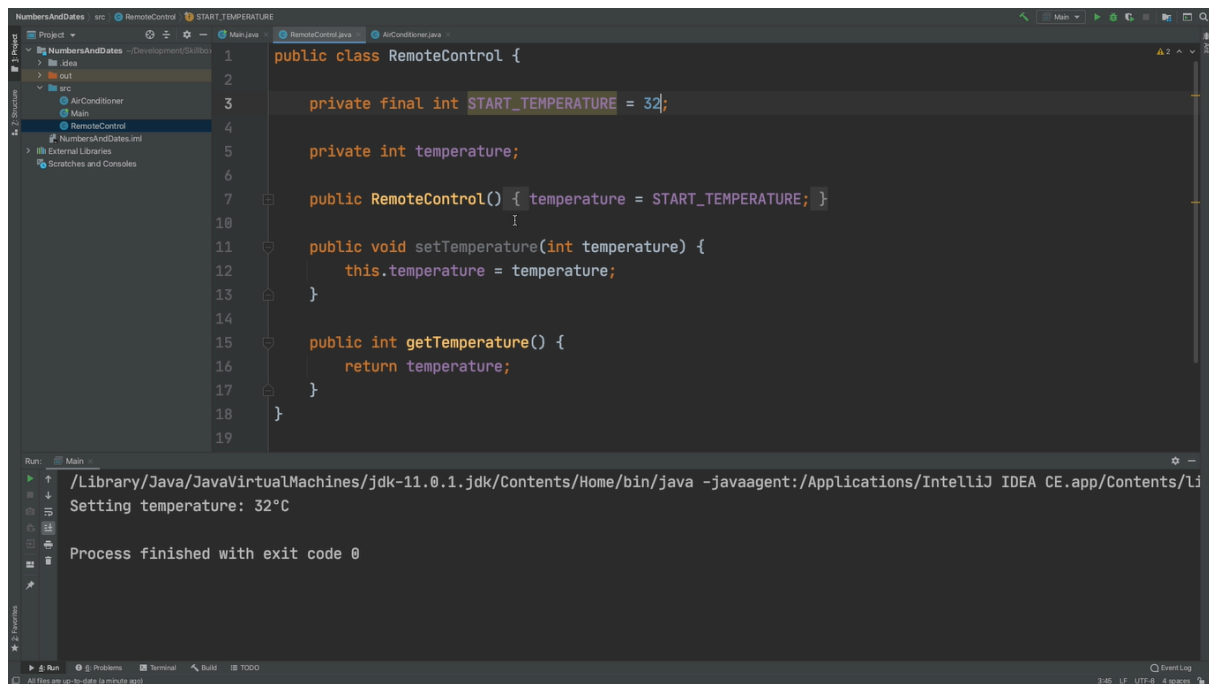


Тип, к которому приводим переменную "doubleValue"

03:07–11:39

Неявное преобразование

Теперь представим, что всё наоборот — с помощью пульта вы можете установить только целочисленное значение.



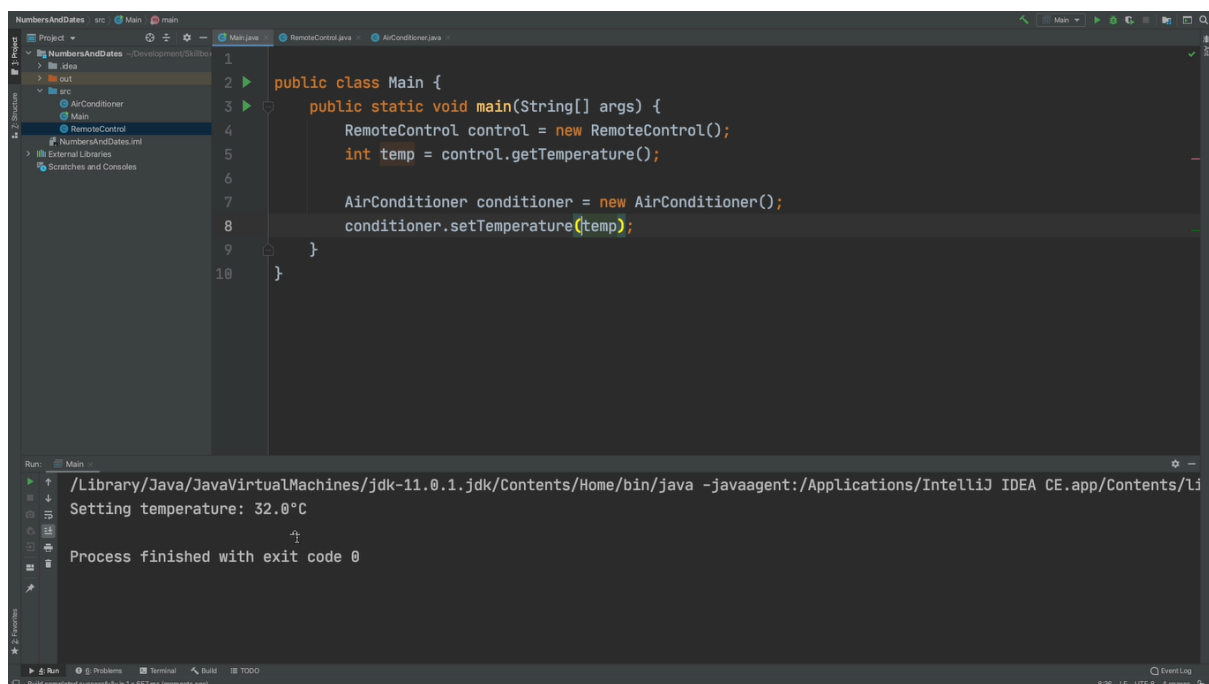
```
1 public class RemoteControl {
2
3     private final int START_TEMPERATURE = 32;
4
5     private int temperature;
6
7     public RemoteControl() { temperature = START_TEMPERATURE; }
8
9     public void setTemperature(int temperature) {
10         this.temperature = temperature;
11     }
12
13     public int getTemperature() {
14         return temperature;
15     }
16 }
17
18
19
```

Run: Main

/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
Setting temperature: 32°C

Process finished with exit code 0

Кондиционер будет принимать в качестве параметра температуры типа double — double temperature. Что же произойдёт в коде? В коде преобразование будет лишним, и мы можем число int передать в метод, который принимает в качестве параметра тип double. Обратите внимание, что данный код верный, потому что происходит **неявное** преобразование типов, то есть число int неявно преобразуется в число double:



```
1 public class Main {
2     public static void main(String[] args) {
3         RemoteControl control = new RemoteControl();
4         int temp = control.getTemperature();
5
6         AirConditioner conditioner = new AirConditioner();
7         conditioner.setTemperature(temp);
8     }
9 }
10
```

Run: Main

/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
Setting temperature: 32.0°C

Process finished with exit code 0

Неявное преобразование — это такое преобразование типов, которое происходит автоматически.

Преобразование чисел разных типов

Явное преобразование:

```
int value = (int) doubleValue;
```

Неявное преобразование:

```
double value = 56;
```

↑
Число типа "int"

Давайте посмотрим ещё на несколько примеров явных и неявных преобразований.

Например, напишем:

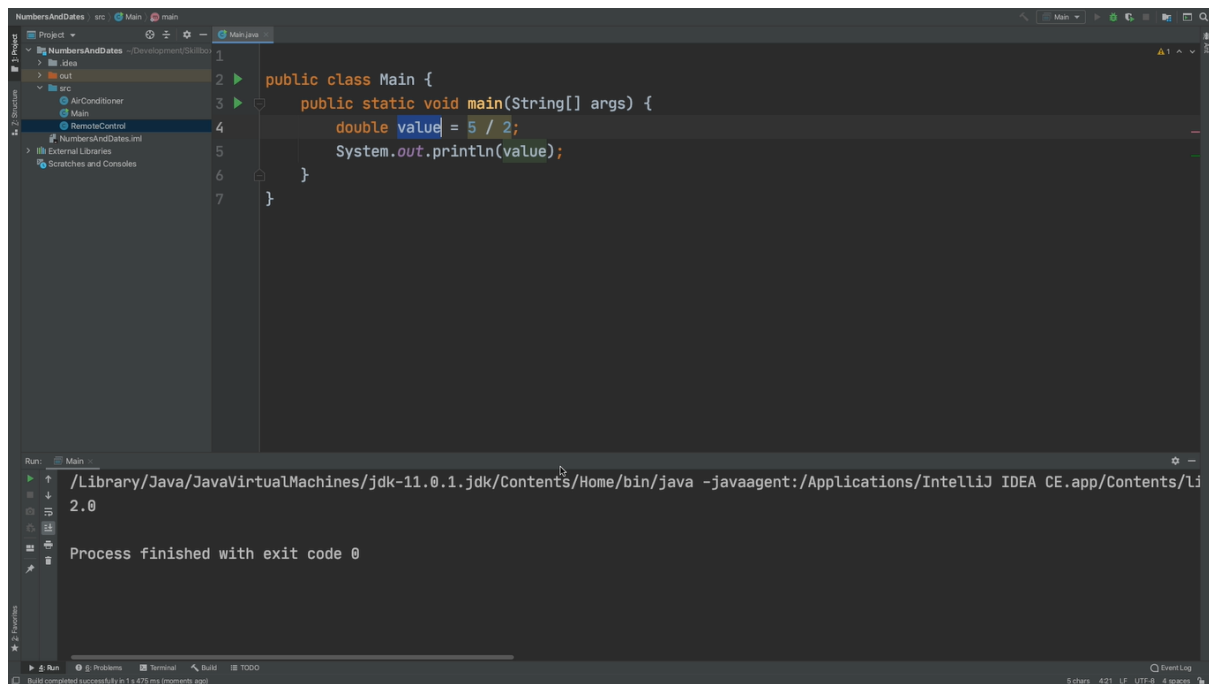
```
double value = 56;
```

Здесь целое число 56, число типа `int`, будет неявным образом преобразовано к типу `double`.

Или напишем так:

```
double value = 56;
```

Происходит деление одного целого числа на другое целое число. Вы уже знаете, что при делении целых чисел друг на друга получается целое число. Вычисляется, сколько делителей умещается в делимом. В данном случае умещается всего две штуки, если мыслить только категориями целых чисел. Получившийся результат, то есть число 2, будет автоматически неявным образом преобразован к числу типа `double`:

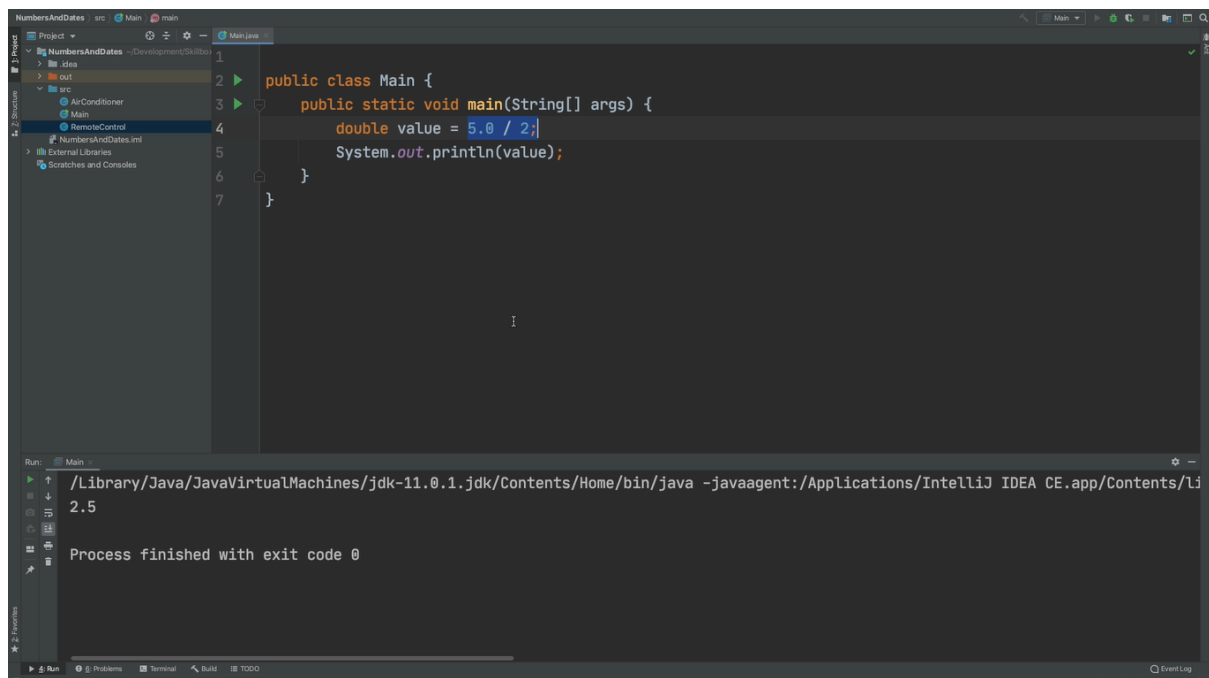


The screenshot shows the IntelliJ IDEA IDE with a project named 'NumbersAndDates'. The 'Main.java' file is open, showing the following code:

```
1 public class Main {
2     public static void main(String[] args) {
3         double value = 5 / 2;
4         System.out.println(value);
5     }
6 }
7
```

The Run window at the bottom shows the command executed: `/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/lib`. The output is `2.0`, and the process finished with exit code 0.

Если хотим, чтобы это вычисление происходило точно, можно указать в явном виде, что это число с плавающей точкой — 5.0. В этом случае число 2 будет преобразовано в число double автоматически:

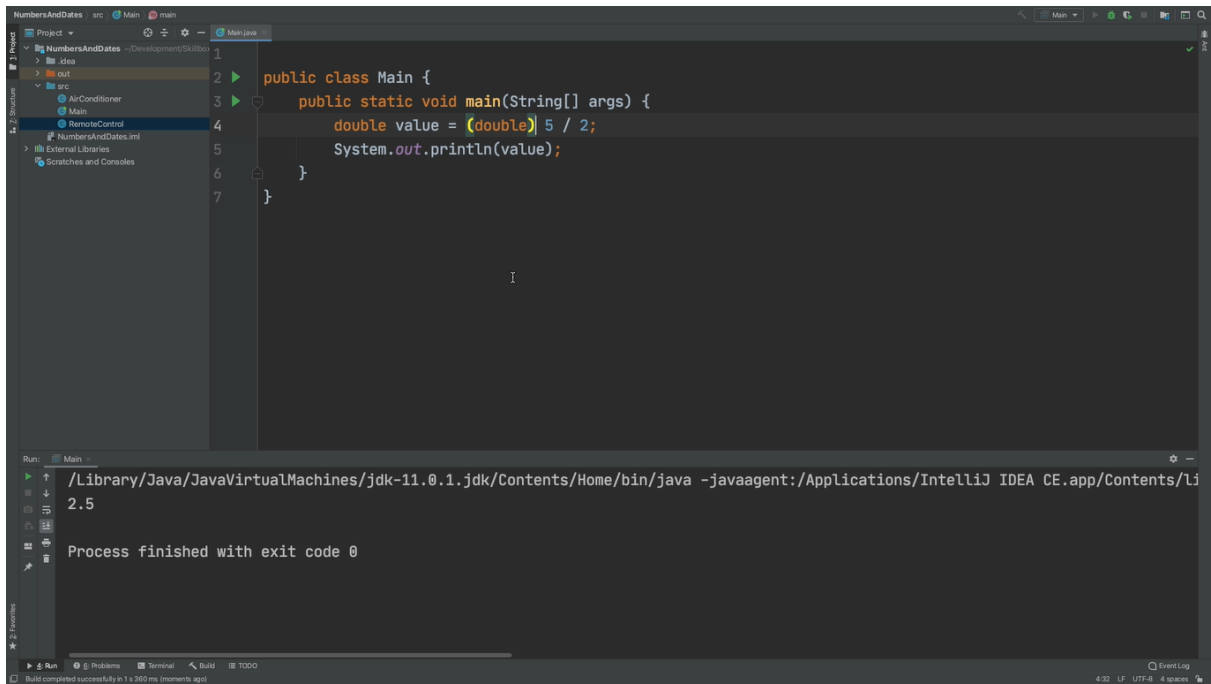


The screenshot shows the IntelliJ IDEA IDE with the same project 'NumbersAndDates'. The 'Main.java' file is open, showing the following code:

```
1 public class Main {
2     public static void main(String[] args) {
3         double value = 5.0 / 2;
4         System.out.println(value);
5     }
6 }
7
```

The Run window at the bottom shows the same command as before. The output is `2.5`, and the process finished with exit code 0.

Также мы можем преобразовать это число явным образом с помощью скобок:



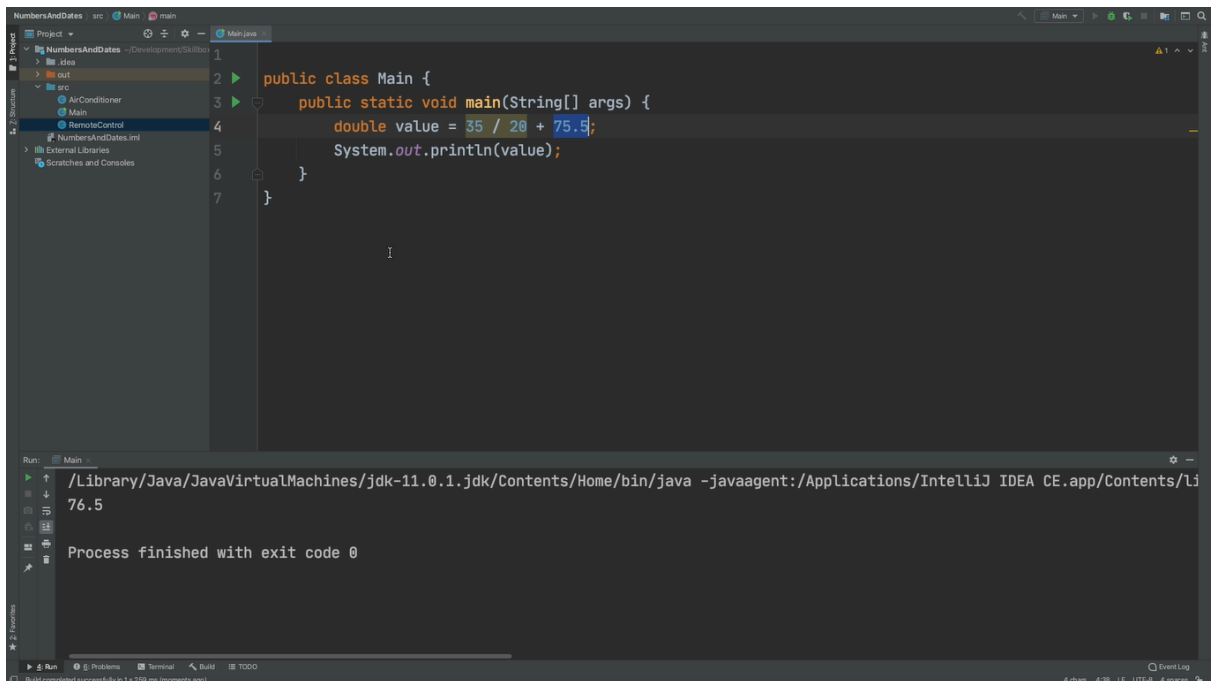
The screenshot shows the IntelliJ IDEA IDE with a project named "NumbersAndDates". The "Main.java" file is open, containing the following code:

```
1 public class Main {  
2     public static void main(String[] args) {  
3         double value = (double) 5 / 2;  
4         System.out.println(value);  
5     }  
6 }  
7
```

The Run window at the bottom shows the command executed: `/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Lj` and the output: `2.5`. The process finished with exit code 0.

Вот ещё один пример, который проиллюстрирует, в какой момент происходит преобразование: $35 / 20 + 75,5$.

Сначала происходит деление. Так как это деление целого числа на целое число, результат будет 1. Затем число 1 будет преобразовано в число типа `double` — `1.0`. Итоговый результат будет `76,5`:



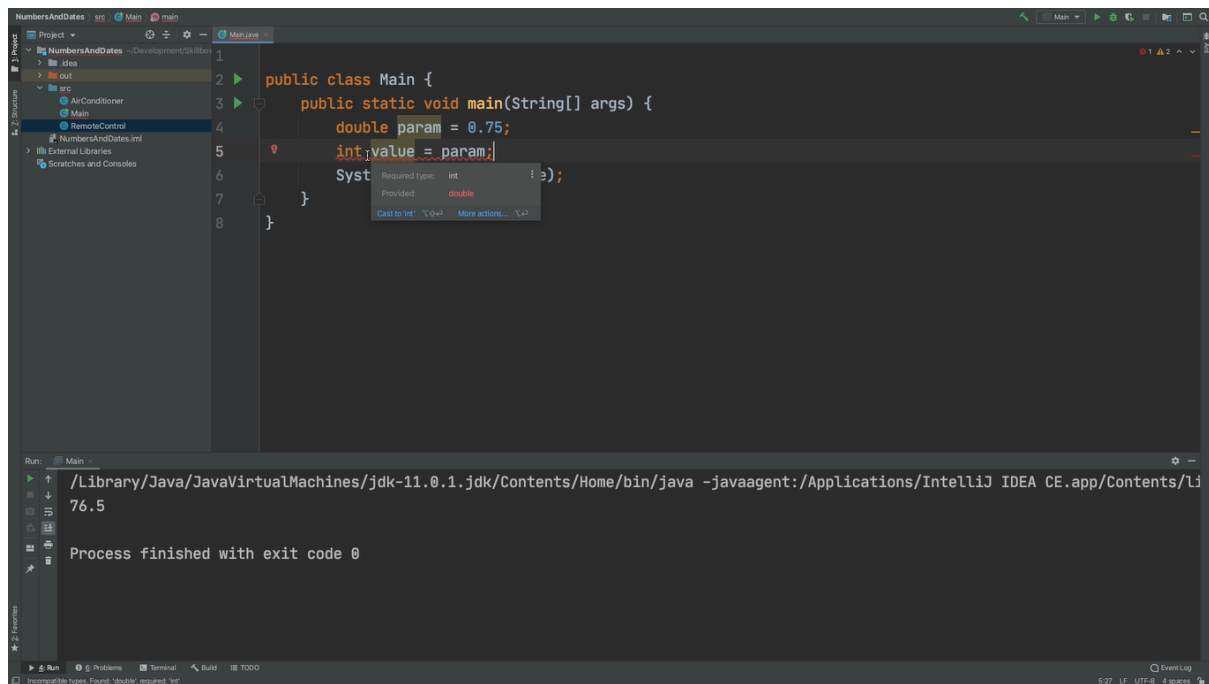
The screenshot shows the IntelliJ IDEA IDE with the same project "NumbersAndDates". The "Main.java" file is open, containing the following code:

```
1 public class Main {  
2     public static void main(String[] args) {  
3         double value = 35 / 20 + 75.5;  
4         System.out.println(value);  
5     }  
6 }  
7
```

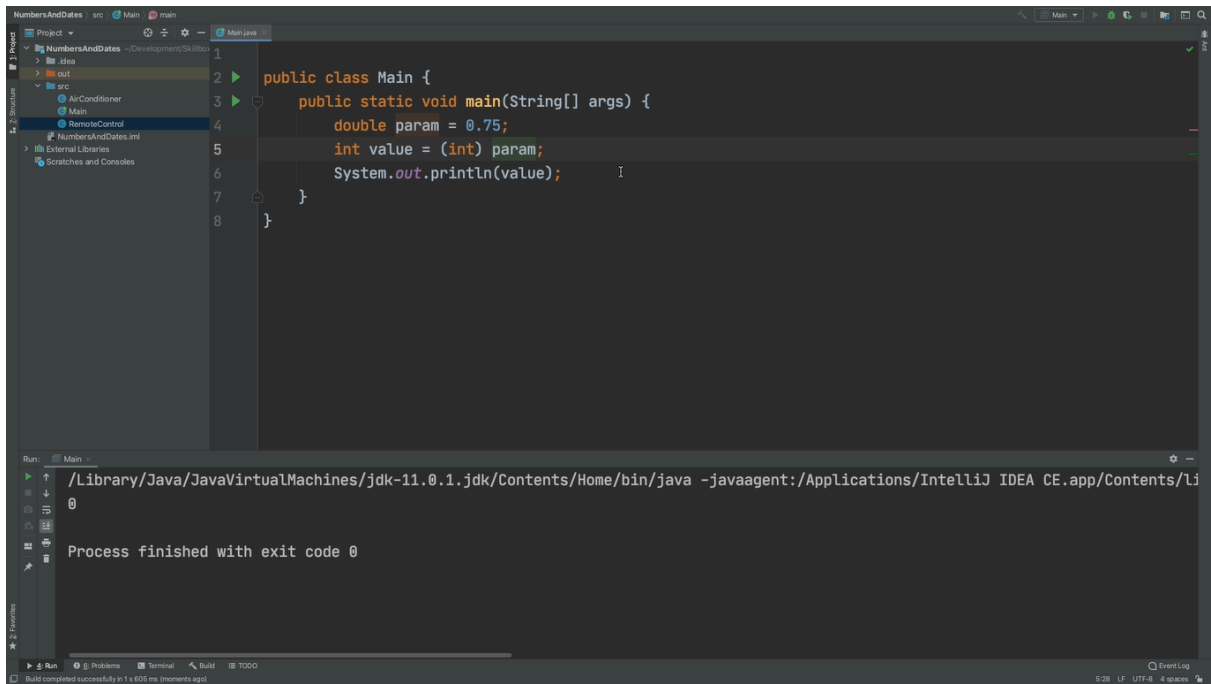
The Run window at the bottom shows the command executed: `/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Lj` and the output: `76.5`. The process finished with exit code 0.

Вот так работает неявное преобразование чисел одного типа в числа другого типа.

Теперь разберём ситуацию, в которой без явного преобразования типов никак не обойтись. Допустим, у вас есть некий параметр 0,75, и вам нужно получить некое целое число из этого параметра. Если вы так напишете, произойдёт ошибка — код будет подчёркнут красным:



Такое преобразование можно сделать только в явном виде, используя круглые скобки. В данном случае значение value будет равно 0.



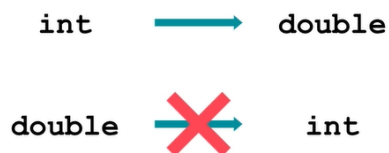
```
1 public class Main {
2     public static void main(String[] args) {
3         double param = 0.75;
4         int value = (int) param;
5         System.out.println(value);
6     }
7 }
8 }
```

Run: Main
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
0
Process finished with exit code 0

Принцип состоит в том, что неявное преобразование происходит только в сторону увеличения размерности, то есть число `int` в число `double` будет преобразовано неявно, а наоборот — нет.

Неявное преобразование

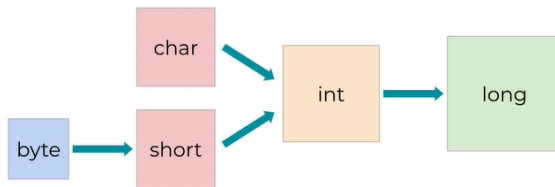
Происходит только в сторону
увеличения размерности



Так происходит потому, что маленькое число в большое преобразовать можно всегда, оно туда всегда влезет, а наоборот не всегда.

Давайте посмотрим, какие неявные преобразования могут происходить между числовыми типами:

Skillbox



Здесь вы видите варианты преобразования между целыми числами. Целые числа от меньшей размерности будут автоматически, то есть неявно, преобразовываться в целые числа большей размерности.

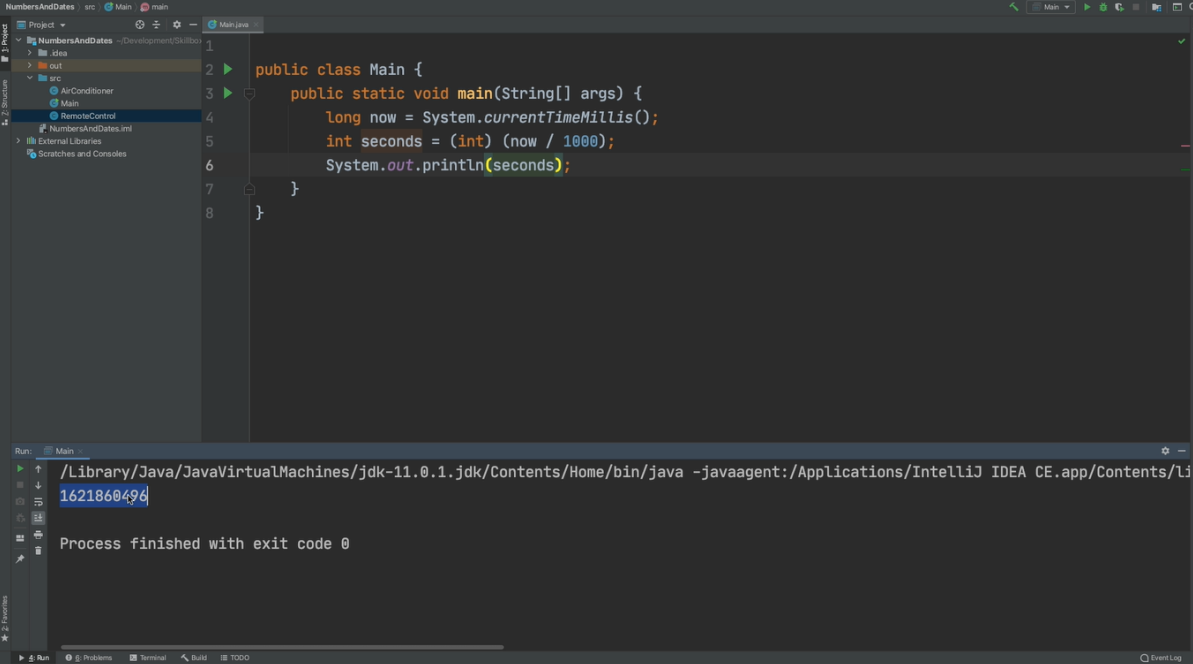
Давайте посмотрим на пример того, как в Java можно и нельзя работать с числами.

К примеру, вам нужно получить текущее время в виде количества секунд. У класса `System` есть для этого специальный метод `currentTimeMillis` — он возвращается число в виде количества миллисекунд, прошедших с 1 января 1970 года до момента вызова этого метода. Возвращается это число в виде числа `long`.

Создадим переменную `long now`, которая будет равна `System.currentTimeMillis`. К примеру, вам нужно получить текущее время, но не в миллисекундах, а в секундах.

Это число вполне себе влезет в переменную типа `int`, и в ней вы можете просто `now` поделить на тысячу. Но такое выражение не сработает, поскольку `now` имеет тип `long` и `long` автоматически, то есть, неявным образом, преобразовываться в число `int` не будет.

Здесь нужно преобразовать итоговый результат к типу `int` в явном виде и получить такой результат:



```
1 public class Main {
2
3     public static void main(String[] args) {
4         long now = System.currentTimeMillis();
5         int seconds = (int) (now / 1000);
6         System.out.println(seconds);
7     }
8 }
```

Run: Main

/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
1621860496

Process finished with exit code 0

Важно: при такой операции преобразуемое число должно обязательно влезть в новый тип, иначе результат может быть очень странным. К примеру, у вас есть число `long`, которое имеет значение триллион. Триллион в число `int` не влезет, поэтому нужно обязательно в конце указать `L`. Попробуем преобразовать число типа `long` в явном виде. Результат будет совершенно не похож на исходное значение:

```
NumbersAndDates  src  Main  main
Project
  NumbersAndDates
    src
      Main
      RemoteControl
    NumbersAndDates.iml
    External Libraries
    Scratches and Consoles
Main.java
1
2 public class Main {
3     public static void main(String[] args) {
4         long value = 1000000000000L;
5         System.out.println((int) value);
6     }
7 }

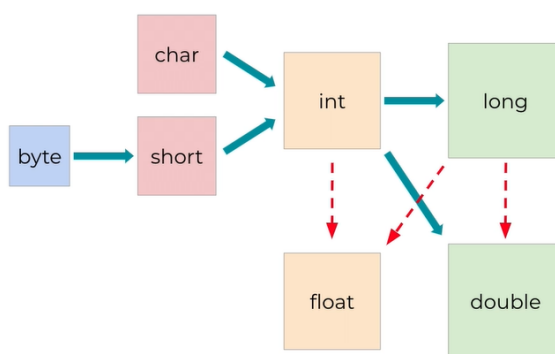
Run: Main
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
-727379968

Process finished with exit code 0
```

Поэтому важно понимать, что явное преобразование может быть опасным, если вы не контролируете размеры значений.

Что касается чисел с плавающей точкой, тут ситуация чуть сложнее: `int` в `double` будет преобразовываться без проблем, это мы уже видели, а вот `int` `long` во `float` или `int` и `long` в `double` будут преобразовываться с потерей точности. Такие преобразования на слайде отмечены красными пунктирными стрелками.

Skillbox



- без потери точности
- - с потерей точности

Посмотрим на примере. У вас есть целое число, допустим, 987654321, и вы его преобразуете в число типа float неявным образом. Выводим это число. Обратите внимание, что произошла потеря точности:

The screenshot shows an IDE with a Java project named 'NumbersAndDates'. The 'Main' class is selected in the project view. The code in the editor is as follows:

```

1 public class Main {
2     public static void main(String[] args) {
3         int number = 987654321;
4         float value = number;
5         System.out.println(value);
6     }
7 }
8

```

The 'Run' window at the bottom shows the execution output:

```

Run: Main
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/lib
98765434E8
Process finished with exit code 0

```

Даже если мы преобразуем число `value` в число типа `int`, обратно точность мы уже не вернём:

The screenshot shows an IDE with a project named 'NumbersAndDates'. The 'src' folder contains files 'AirConditioner', 'Main', and 'RemoteControl'. The 'Main.java' file is open, showing the following code:

```

1
2 public class Main {
3     public static void main(String[] args) {
4         int number = 987654321;
5         float value = number;
6         System.out.println((int) value);
7     }
8 }

```

The 'Run' tab at the bottom shows the execution output:

```

Run: Main
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
987654336

Process finished with exit code 0

```

К вопросу точности чисел с плавающей точкой мы ещё вернёмся отдельно и рассмотрим этот вопрос подробно.

11:39–12:08

Выводы

Итак, в этой теме вы познакомились с явным и неявным преобразованием типов. Для явного преобразования вам нужно указать необходимый тип в круглых скобках, но при этом нужно быть осторожными.

Также вы узнали, что неявное преобразование происходит автоматически и только в сторону увеличения размерности.

Преобразование в числа с плавающей точкой может происходить с потерей точности, о которой поговорим в следующей теме.